Cyber Shujaa

DATA & AI PROGRAMME

Assignment Submission Template

1. TITLE & OBJECTIVE

Title: Classification Models

LINK: https://colab.research.google.com/drive/1ic7ARrQCtdXxZ7dxilRpomt5WIA0m82Y?usp=sharing

GITHUB: https://github.com/sangvivy/wine-quality-classification-ml-model-comparison

Objective: To build and evaluate six machine learning classification models to predict wine classes using the Wine dataset from Scikit-learn.

2. SETUP / IMPORTS

The project was implemented in Python using Jupyter Notebook or Google Colab. The Wine dataset was loaded from sklearn.datasets using load_wine().

Libraries used:

- pandas and numpy for data handling.
- scikit-learn for dataset loading, preprocessing, model building, and evaluation.

The dataset contains 178 samples, 13 numerical features, and three target classes.

3. MAIN WORK

Step	Task	Work Performed
1	Load dataset	Loaded the Wine dataset using load_wine() and converted the features into a Pandas DataFrame.
2	Explore data (EDA)	Examined the dataset shape, data types, missing values, descriptive statistics, and target-class distribution.
3	Prepare data	Separated the 13 input features from the target variable and split the dataset into 80% training data and 20% testing data using stratified sampling.
4	Scale features	Applied StandardScaler to standardize features for Logistic Regression, KNN, and SVM. The scaler was fitted on training data only.

5	Build models	Trained Logistic Regression, Decision Tree, Random Forest, K-Nearest Neighbours (KNN), Naive Bayes, and Support Vector Machine (SVM).
6	Evaluate models	Calculated accuracy, precision, recall, F1-score, and confusion matrices for all six classifiers using the test set.
7	Compare models	Created a comparison table ranking the classifiers according to their test accuracy.

4. RESULTS & INTERPRETATION

Loading the data

```
from sklearn.datasets import load_wine
import pandas as pd
import numpy as np

# Load the Wine dataset
wine = load_wine()

# Convert features into a DataFrame
df = pd.DataFrame(wine.data, columns=wine.feature_names)

# Display the first five rows
df.head()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavanoid_phenols
0	14.23	1.71	2.43	15.6	127.0	2.80	3.06	0.28
1	13.20	1.78	2.14	11.2	100.0	2.65	2.76	0.26
2	13.16	2.36	2.67	18.6	101.0	2.80	3.24	0.30
3	14.37	1.95	2.50	16.8	113.0	3.85	3.49	0.24
4	13.24	2.59	2.87	21.0	118.0	2.80	2.69	0.39

Next steps: Generate code with df | New interactive sheet

Exploratory Data Analysis

```python
# Display the number of rows and columns
print("Dataset shape:", df.shape)

# Display column names and data types
df.info()
```

```
Dataset shape: (178, 13)
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 178 entries, 0 to 177
Data columns (total 13 columns):
 #   Column                        Non-Null Count  Dtype
---  ------                        --------------  -----
 0   alcohol                       178 non-null    float64
 1   malic_acid                    178 non-null    float64
 2   ash                           178 non-null    float64
 3   alcalinity_of_ash             178 non-null    float64
 4   magnesium                     178 non-null    float64
 5   total_phenols                 178 non-null    float64
 6   flavanoids                    178 non-null    float64
 7   nonflavanoid_phenols          178 non-null    float64
 8   proanthocyanins               178 non-null    float64
 9   color_intensity               178 non-null    float64
 10  hue                           178 non-null    float64
 11  od280/od315_of_diluted_wines  178 non-null    float64
 12  proline                       178 non-null    float64
dtypes: float64(13)
memory usage: 18.2 KB
```

Interpretation: The Wine dataset contains 178 samples and 13 numerical features, with all values present and no missing data. All features use the float64 data type, making them suitable for machine learning. The dataset is relatively small and contains chemical measurements that can be used to classify wine samples into three target classes.

Step 2B: Examine the basic statistics

```python
# Display descriptive statistics
df.describe()
```

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium	total_phenols	flavanoids	nonflavar
count	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	178.000000	
mean	13.000618	2.336348	2.366517	19.494944	99.741573	2.295112	2.029270	
std	0.811827	1.117146	0.274344	3.339564	14.282484	0.625851	0.998859	
min	11.030000	0.740000	1.360000	10.600000	70.000000	0.980000	0.340000	
25%	12.362500	1.602500	2.210000	17.200000	88.000000	1.742500	1.205000	
50%	13.050000	1.865000	2.360000	19.500000	98.000000	2.355000	2.135000	
75%	13.677500	3.082500	2.557500	21.500000	107.000000	2.800000	2.875000	
max	14.830000	5.800000	3.230000	30.000000	162.000000	3.880000	5.080000	

Interpretation: The descriptive statistics show that the 13 features have different numerical ranges and levels of variation. For example, alcohol has a mean of 13.00, while proline has a mean of 746.89. Proline and magnesium also show considerable variability, as indicated by their standard deviations. These differences suggest that feature scaling will be important for models such as Logistic Regression, KNN, and SVM.

Step 2C: Explore the target classes

```
# Add the target class to the dataset
df["target"] = wine.target

# Count samples in each class
print(df["target"].value_counts().sort_index())

# Display the class names
print(wine.target_names)
```

```
target
0    59
1    71
2    48
Name: count, dtype: int64
['class_0' 'class_1' 'class_2']
```

Interpretation: The dataset contains three target classes: class 0 has 59 samples, class 1 has 71 samples, and class 2 has 48 samples. The classes are relatively balanced, although class 1 has the most samples and class 2 has the fewest. This distribution is suitable for training and evaluating a multiclass classification model.

Step 3: Prepare the data

Step 3A: Separate features and target

```
# Separate input features and target labels
X = df.drop("target", axis=1)
y = df["target"]

print("Features shape:", X.shape)
print("Target shape:", y.shape)
```

```
Features shape: (178, 13)
Target shape: (178,)
```

Interpretation: The dataset was separated into 13 input features ((X)) and one target variable ((y)), which represents the wine classes.

Step 3B: Split into training and testing sets

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.2,
    random_state=42,
    stratify=y
)

print("Training set:", X_train.shape)
print("Testing set:", X_test.shape)
```

```
Training set: (142, 13)
Testing set: (36, 13)
```

Interpretation: The data was divided into 142 training samples (80%) and 36 testing samples (20%). Stratified sampling was used to maintain the class proportions in both sets.

Step 3C: Scale the features

[7] ✓ 0s

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

print("Scaled training set:", X_train_scaled.shape)
print("Scaled testing set:", X_test_scaled.shape)
```

```
Scaled training set: (142, 13)
Scaled testing set: (36, 13)
```

Interpretation: StandardScaler was fitted on the training data and applied to both the training and testing sets to standardize the features while preventing data leakage.

Step 4: Build the six classification models

[8] ✓ 2s

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.naive_bayes import GaussianNB
from sklearn.svm import SVC

# Initialize the six classification models
models = {
    "Logistic Regression": LogisticRegression(max_iter=10000, random_state=42),
    "Decision Tree": DecisionTreeClassifier(random_state=42),
    "Random Forest": RandomForestClassifier(random_state=42),
    "KNN": KNeighborsClassifier(),
    "Naive Bayes": GaussianNB(),
    "SVM": SVC()
}

# Train each model
for name, model in models.items():
    if name in ["Logistic Regression", "KNN", "SVM"]:
        model.fit(X_train_scaled, y_train)
    else:
        model.fit(X_train, y_train)

    print(f"{name} trained successfully.")
```

Step 5A: Evaluate all six models

```python
from sklearn.metrics import accuracy_score, classification_report, confusion_matrix

# Store model evaluation results
results = {}

for name, model in models.items():
    # Use scaled test data for scale-sensitive models
    if name in ["Logistic Regression", "KNN", "SVM"]:
        X_eval = X_test_scaled
    else:
        X_eval = X_test

    # Generate predictions
    y_pred = model.predict(X_eval)

    # Calculate accuracy
    accuracy = accuracy_score(y_test, y_pred)

    # Store results
    results[name] = {
        "Accuracy": accuracy,
        "Predictions": y_pred
    }
```

```python
    print(f"\n{'=' * 50}")
    print(name)
    print(f"Accuracy: {accuracy:.4f}")

    print("\nClassification Report:")
    print(classification_report(
        y_test,
        y_pred,
        target_names=wine.target_names,
        zero_division=0
    ))

    print("Confusion Matrix:")
    print(confusion_matrix(y_test, y_pred))
```

```
--------------------------------------------------
Logistic Regression
Accuracy: 0.9722

Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       0.93      1.00      0.97        14
     class_2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36

Confusion Matrix:
[[12  0  0]
 [ 0 14  0]
 [ 0  1  9]]
```

Logistic Regression: The model achieved 97.22% accuracy, correctly classifying 35 out of 36 wine samples. It classified classes 0 and 1 perfectly in terms of recall, but misclassified one sample from class 2 as class 1.

```
==================================================
Decision Tree
Accuracy: 0.9444

Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      0.92      0.96        12
     class_1       0.88      1.00      0.93        14
     class_2       1.00      0.90      0.95        10

    accuracy                           0.94        36
   macro avg       0.96      0.94      0.95        36
weighted avg       0.95      0.94      0.94        36

Confusion Matrix:
[[11  1  0]
 [ 0 14  0]
 [ 0  1  9]]
```

Decision Tree: The model achieved 94.44% accuracy, correctly classifying 34 out of 36 samples. It misclassified one class 0 sample as class 1 and one class 2 sample as class 1.

```
==================================================
Random Forest
Accuracy: 1.0000

Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       1.00      1.00      1.00        14
     class_2       1.00      1.00      1.00        10

    accuracy                           1.00        36
   macro avg       1.00      1.00      1.00        36
weighted avg       1.00      1.00      1.00        36

Confusion Matrix:
[[12  0  0]
 [ 0 14  0]
 [ 0  0 10]]
```

Random Forest: The model achieved 100% accuracy, correctly classifying all 36 test samples. Its precision, recall, and F1-score were all 1.00 across the three classes, indicating perfect performance on this test set.

```
==================================================
KNN
Accuracy: 0.9722

Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       1.00      0.93      0.96        14
     class_2       0.91      1.00      0.95        10

    accuracy                           0.97        36
   macro avg       0.97      0.98      0.97        36
weighted avg       0.97      0.97      0.97        36

Confusion Matrix:
[[12  0  0]
 [ 0 13  1]
 [ 0  0 10]]

--------------------------------------------------
```

K-Nearest Neighbours (KNN): The model achieved 97.22% accuracy, correctly classifying 35 out of 36 samples. It misclassified one class 1 sample as class 2, while correctly identifying all class 0 and class 2 samples.

```
==================================================
Naive Bayes
Accuracy: 0.9722

Classification Report:
              precision    recall  f1-score   support

     class_0       0.92      1.00      0.96        12
     class_1       1.00      0.93      0.96        14
     class_2       1.00      1.00      1.00        10

    accuracy                           0.97        36
   macro avg       0.97      0.98      0.97        36
weighted avg       0.97      0.97      0.97        36

Confusion Matrix:
[[12  0  0]
 [ 1 13  0]
 [ 0  0 10]]
```

Naive Bayes: The model achieved 97.22% accuracy, correctly classifying 35 out of 36 samples. It misclassified one class 1 sample as class 0, while correctly identifying all class 2 samples.

```
==================================================
SVM
Accuracy: 0.9722

Classification Report:
              precision    recall  f1-score   support

     class_0       1.00      1.00      1.00        12
     class_1       0.93      1.00      0.97        14
     class_2       1.00      0.90      0.95        10

    accuracy                           0.97        36
   macro avg       0.98      0.97      0.97        36
weighted avg       0.97      0.97      0.97        36

Confusion Matrix:
[[12  0  0]
 [ 0 14  0]
 [ 0  1  9]]
```

Support Vector Machine (SVM): The model achieved 97.22% accuracy, correctly classifying 35 out of 36 samples. Like Logistic Regression, it misclassified one class 2 sample as class 1.

Step 6: Compare the models

```
# Create a model comparison table
comparison = pd.DataFrame({
    "Model": list(results.keys()),
    "Accuracy": [results[name]["Accuracy"] for name in results]
})

comparison["Accuracy (%)"] = comparison["Accuracy"] * 100

comparison = comparison.sort_values(
    by="Accuracy",
    ascending=False
).reset_index(drop=True)

comparison[["Model", "Accuracy (%)"]]
```

	Model	Accuracy (%)
0	Random Forest	100.000000
1	Logistic Regression	97.222222
2	Naive Bayes	97.222222
3	KNN	97.222222
4	SVM	97.222222
5	Decision Tree	94.444444

Interpretation:

Random Forest performed best, achieving 100% accuracy by correctly classifying all 36 test samples. Logistic Regression, Naive Bayes, KNN, and SVM followed with 97.22% accuracy each, while Decision Tree had the lowest accuracy at 94.44%. Random Forest may have performed best because it combines multiple decision trees, helping capture complex relationships in the data and reducing the impact of individual tree errors. However, cross-validation would be useful to confirm whether its performance is consistent across different data splits.

5. CONCLUSION

The project involved loading and exploring the Wine dataset, preparing and scaling the features, training six machine learning classifiers, and evaluating their performance. Random Forest was the best-performing model on the test set, achieving 100% accuracy. The findings demonstrate how machine learning can classify wine samples using chemical measurements.